

Instituto Superior de Engenharia de Lisboa
Bachelor in Computer Science and Computer Engineering
Bachelor in Informatics, Networks and Telecommunications Engineering
Internet Programming/Introduction to Web Programming
Winter Semester of 2025/2026 – **2nd practical assignment - FOCCACIA - Part1**

PHASE 1

Due date for this assignment: 23/11/2025-23h59

Delivery method

The delivery method is done through Github Classroom, using the invitation links below. Following the corresponding link, login with your github user and select your student id to make the correspondence between them and create a new group or join an existing one. If you create a new repository, the repository name should be: `<LEIC|LEIRT>2526i-<COURSE><CLASS>-<GROUP_NUMBER>`. For example:

- Group 2 for class LEIC31D from IPW course, should name the repository: `LEIC2526i-IPW31D-G02`
- Group 1 for class LEIRT51D from PI course `LEIRT2526i-PI51D-G01`.

The registration in the assignment should be done through the following Github Classroom (<https://classroom.github.com/a/eV3H1acA>)

The assignment should be delivered in this repository, by creating a tag called `FOCCACIA-P1`. If you need to make some changes to the delivery, just create another tag named `FOCCACIA-P1.1`, `FOCCACIA-P1.1`, etc.

Introduction

The practical component evaluation for this course will be carried out based on the FOCCACIA (FOotball Complete Clubs Api and Chelas Internet Application), to be developed throughout the semester. This application provides access, through a web interface (hypermedia), using the information provided by the [API Football](#). To access this API, each student must have the API key that identifies the student application's requests (obtain by email after sign in). That key must be included in the header X-Auth-Token of each HTTP request. Note that this is a free API, and therefore it has [rate limits](#) for your requests. Be sure you understand and comply with these limits.

The development will be carried out incrementally, necessarily involving several code refactoring cycles and, therefore, it is essential that you make use of good programming practices in general, in order to reduce the effort associated with each cycle.

The development of the FOCCACIA application has 3 development cycles, phased in three parts (Part 1, Part 2 and Part 3). For each one, the deadline for delivering the solution will be defined, and it will be a non-negotiable requirement.

For each FOCCACIA functionality, the corresponding HTTP *endpoint- must be defined. The description of the application API (i.e. all application endpoints) must appear on the repository in an [OpenAPI](#) file, named `docs/foccacia-api-spec.json` (or .yaml). The repository must also contain REST client compatible files

(with .http extension), with the requests that validate the API, in a file named `docs/foccacia-api-test.http`.

Summary of the artifacts to be submitted upon delivery:

- OpenAPI/Swagger file with API documentation - `docs/foccacia-api-spec.[yaml|json]`
- Http files compatible with [REST Client Visual Studio Code Plugin](#) - `docs/foccacia-api-requests.http`
- Node.js FOCCACIA application files

Functional Requirements

Develop a web application that provides a web API that follows current principals of current Web the REST principles, with responses in JSON format and that supports the following features:

- Get the available competitions. For each competition it must return the code and name.
- Get teams for a specific competition and season. For each team, it must contain the team name, country and the name and position for all its players.
- Favorite "Best XI" Groups - Each group represents a user's selection of their favorite players from a specific competition and year.
 - Create group providing its name, description, competition and year
 - Edit group by changing its name and description
 - List all groups
 - Delete a group
 - Get group details, returns:
 - Group `name`
 - Group `description`
 - `competition` and `year`
 - List of selected players including:
 - `playerId`
 - `playerName`
 - `teamId`
 - `teamName`
 - `position`
 - `nationality`
 - `age`
 - Add a player to a group's Best XI, for each player added, store:
 - `playerId`
 - `playerName`
 - `teamCode`

- `teamName`

NOTE : Each group may have 11 players at maximum, and each player must belong to the year and competition of the group.

- Remove a player from a group
- Create new user, given its username For all group operations, a user token must be sent in the [Authorization header](#) using a Bearer Token. This token is generated at user creation, and consists of a UUID string, obtained from the `crypto.randomUUID()` method.

Non-Functional Requirements

The application must be developed with Node.js technology. To handle/receive HTTP requests, the [express](#) module must be used. To make HTTP requests, the [fetch](#) global function used.

The [API Football](#) must be used to obtain data (query) about the teams. Here follow some resources that may be used in the implementation. These are just suggestions. Each group may use other resources available in the API.

- Get all competitions: <http://api.football-data.org/v4/competitions>)
- Get the details of a competition for a specific season: <http://api.football-data.org/v4/competitions/<CompetitionCode>/teams?season=<year>>)
- Get the details of a team for a specific season: <http://api.football-data.org/v4/competitions/teams/<teamId>?season=<year>>)

The data that is specific to the application, which can be created, altered and deleted, namely the entire group management, in this phase must be stored in memory.

Any of the base modules of Node.js can be used. In addition to these, in this 1st part of the work, only the following modules can be used:

- `express` - Handling HTTP requests
- `debug` – Debug messages (optional)
- `mocha` and optionally `chai` - Unit tests

Any other module you intend to use must be previously discussed and authorized by the corresponding teacher.

All PUT and POST requests must send their data (except unique resource identifiers) in the request body (*body*) and not in the *query string*. The body should be handled by [builtin Express JSON middleware](#).

The server application must consist of at least 5 Node modules:

- `foccacia-server.mjs` - file that constitutes the entry point to the server application
- `foccacia-web-api.mjs` - implementation of the HTTP routes that make up the REST API of the web application
- `foccacia-services.mjs` - implementation of the logic of each of the application's functionalities
- `fapi-teams-data.mjs` - access to the Football API.

- `foccacia-data-mem.mjs` - access to foccacia data (groups and users), in this version stored in memory

The dependencies between these modules are as follows:

```
foccacia-server.mjs -> foccacia-web-api.mjs -> foccacia-services.mjs -> fapi-
teams-data.mjs
                                                    -> foccacia-
data-mem.mjs
```

The server application development methodology must be as follows and in this order:

1. Design and document API routes (HTTP request type + URL+example response content) using OpenAPI/Swagger format.
2. Create HTTP requests in REST Client file (ex. FOCCACIA) to test API routes
3. Implement the server application input module: `foccacia-server.mjs`. For this module it is not necessary to create unit tests, since it must not implement any logic other than receiving some arguments from the command line (configuration), registering routes and starting the web server. This module can be built as each route in `foccacia-web-api.mjs` is implemented.
4. In the `foccacia-web-api.mjs` module, implement the API routes one by one.
 - For each route implemented, use REST Client to verify the correct functioning of that route.
 - Only move on to implementing the next route when the previous one is fully implemented and tested.
 - For each route, create a request in the REST Client file that validates it.
 - In this phase of the implementation of the module `foccacia-web-api.mjs` use local data (*mock- of `foccacia-services.mjs`), that is, testing must be performed without access to the API Football or memory storage.
5. Implement application services in `foccacia-services.mjs` module.
 - Follow an approach similar to the one used in `foccacia-web-api.mjs` in the development of the features of this module and respective unit tests.
 - `foccacia-services.mjs` module unit tests must be run without access to the API Football (`fapi-teams-data.mjs`). That is, create a mock for that access.
6. Implement data access modules:
 - `fapi-teams-data.mjs` - access to the API Football.
 - `foccacia-data-mem.mjs` - access groups data.

Instituto Superior de Engenharia de Lisboa

Bachelor in Computer Science and Computer Engineering

Bachelor in Informatics, Networks and Telecommunications Engineering

Internet Programming/Introduction to Web Programming

Winter Semester of 2025/2026 – **3rd practical assignment - FOCCACIA - Part2**

PHASE 2

Due date for this assignment: 20/12/2025-23h59

Delivery method

The development of the FOCCACIA application has 3 development cycles, phased in three parts. For each one, the deadline for delivering the solution will be defined, and it will be a non-negotiable requirement.

This document includes the requirements of **part 2 of the FOCCACIA application**. The assignment should be delivered in the same Git repository used in part 1, by creating a tag called **FOCCACIA-P2**. If you need to make some changes to the delivery, just create another tag named **FOCCACIA-P2.1**, **FOCCACIA-P2.2**, etc.

The main goals of this part are adding a Web user interface to the FOCCACIA application developed in part 1, storing data in a database instead of memory, and incorporating new technologies and techniques covered in lectures.

Functional requirements

1. Create a web interface for presentation in a *web browser*, for all the functionalities provided by the Web API. All operations that in Web Api are implemented with PUT and DELETE methods, except for user creation, must be implemented using the POST method using HTML forms. This web interface is server-side rendered, using the following technologies: HTML, CSS, and Handlebars. You may use Bootstrap for the base style of the user interface.

When using the web application, in no situation the (human) user will have to know and/or introduce any id for groups or teams or leagues. The only situations in which it is allowed to write the name of a teams is to carry out searches in order to obtain a list of results. The only situation in which it is allowed to manually enter the name of a group is when creating or editing that group.

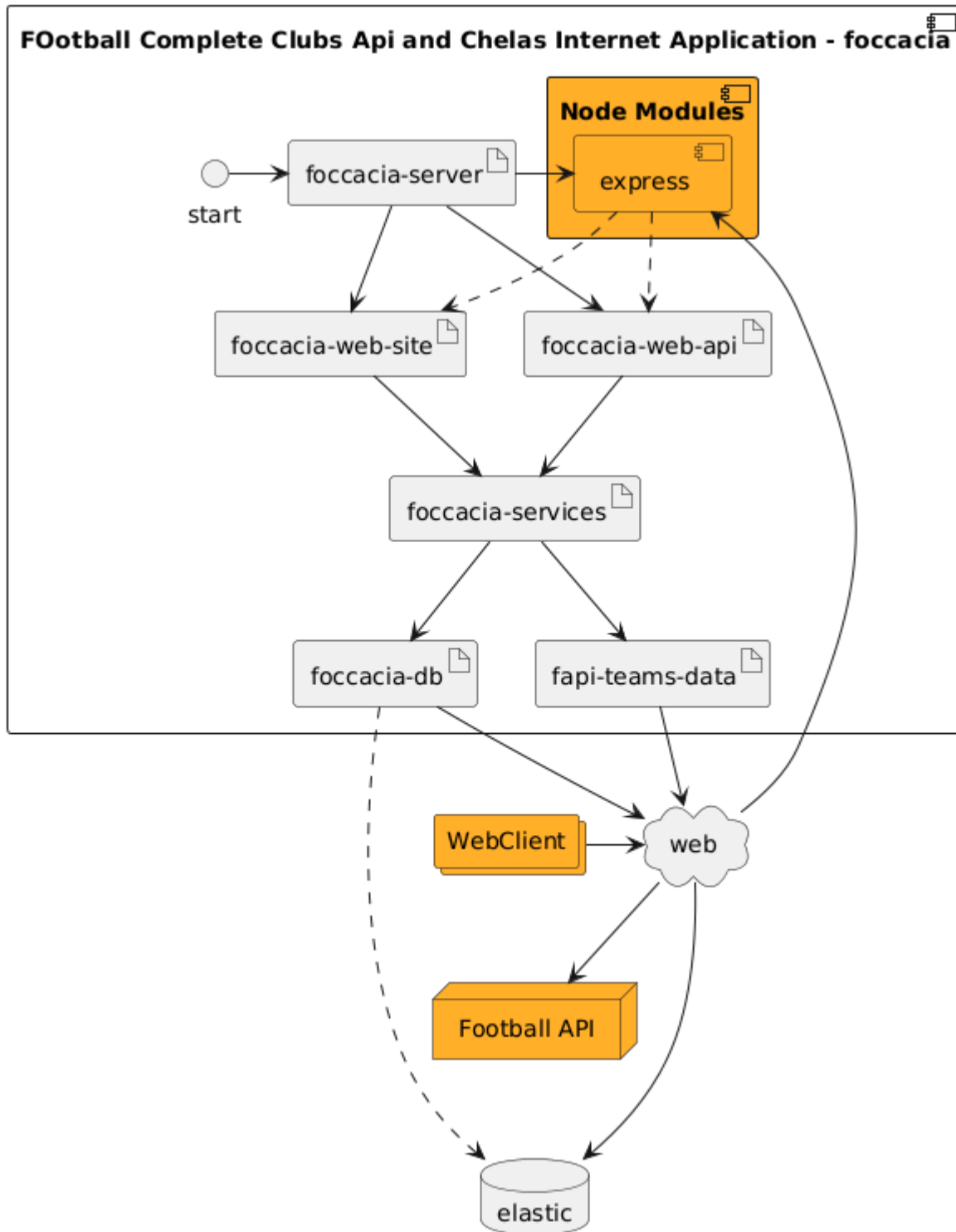
Non-functional requirements

1. The web HTML and CSS interface should be implemented in a new file called **focaccia-web-site.mjs** that should be at the same level as **focaccia-web-api.mjs**.
2. Create a new module that replaces **focaccia-data-mem.mjs** so that the data is stored in an ElasticSearch database. This change should not imply any additional change in the remaining modules of the application besides module loading in **focaccia-server.mjs** (or a specific module for application configurations). The interaction with ElasticSearch must be done through its HTTP API using fetch function, without using any specific node module for ElasticSearch.

NOTE: The module **focaccia-data-mem.mjs** must not be deleted and it should be possible to change the implementations by changing the module imports in the module that configures the application.

3. In addition to the previous requirements, this part of work should be used to improve the code quality as well as the quality and quantity of tests, whether unit or integration.
4. [optional] Create integration tests for the API with **supertest** module

The server application modules dependencies should be as follows:



Instituto Superior de Engenharia de Lisboa

Bachelor in Computer Science and Computer Engineering

Bachelor in Informatics, Networks and Telecommunications Engineering

Internet Programming/Introduction to Web Programming

Winter Semester of 2025/2026 – **4th practical assignment - FOCCACIA - Part3**

PHASE 3

Due date for this assignment: 11/01/2026-23h59.

Delivery method

The development of the FOCCACIA application has 3 development cycles, phased in three parts. For each one, the deadline for delivering the solution will be defined, and it will be a non-negotiable requirement.

This document includes the requirements of **part 3 of the FOCCACIA application**. The assignment should be delivered in the same Git repository used in part 1, by creating a tag called **FOCCACIA-P3**. If you need to make some changes to the delivery, just create another tag named **FOCCACIA-P3.1**, **FOCCACIA-P3.2**, etc.

The main goals of this part are adding authentication to the web application with user interface (web site), and support the PUT and DELETE operations not implemented in the previous phase, using JavaScript on the client side.

Functional requirements

1. Add user registration and authentication functionality to the FOCCACIA application. All event group management features must be accessible only to authenticated users. Groups are private to each user and can only be manipulated by their owner. The authentication must be implemented with the **Passport** module.
2. (optional) All functionalities available in the Web API developed in FOCCACIA part 1, must be available also through the user interface available at the web site. The DELETE and PUT operations available in the Web API may be used by the web site pages to implement the corresponding functionalities, with client JavaScript on the web page using the browser **fetch()** function to make the HTTP requests.

Again, as it was a requirement for part 2, when using the web application, in no situation the (human) user will have to know and/or introduce any id for groups or teams or leagues. The only situations in which it is allowed to write the name of a team is to carry out searches in order to obtain a list of results. The only situation in which it is allowed to manually enter the name of a group is when creating or editing that group.

Non-functional requirements

1. With this part of the work, a report must be delivered to the group's repository wiki, describing the implementation of the full work developed during the semester. Thus, the intermediate states that the implementation went through in each of the phases should not be included. The report must include:
 - Description of the application structure, in both components (server and client).
 - Data storage design (i.e. in Elasticsearch), namely: index or indices; document properties; relations between documents, if exist; etc.
 - Description of the mapping between the Elasticsearch's FOCCACIA documents and the web application objects model.
 - Server API documentation.
 - Instructions for **all** the previous steps that are necessary to run the application and the respective tests.
 - These steps must include the actions necessary for the automatic introduction of test data, so that it is possible to run the application with data.
 - Instructions must have all the information needed to run the application on any machine, namely the teacher's. The instructions to make the application run should be executed in a maximum of 5 min.